

Predicting 24h Weather at Tufts University Using CNN

[Code Repository](#)

Part 1: Training the CNN

1.1 Data Handling

The training and validation data was taken from 2018, 2019, and 2020. Any data points with NA values were dropped, and the remaining data was randomly shuffled for the train/validation split (random seed = 42). We took 80% from the final pool for training and 20% for validation. All the data was normalized before training to make different features align in similar scales. The resulting model was then tested on 2021 data.

1.2 Model Structure

We decided to have a structure modeled after ResNet. The model mainly consists of an initial convolutional layer, 4 residual blocks, a global average pooling layer, one head for continuous values prediction, and one head for binary values prediction. The inputs are normalized into a matrix with dimension (B, C, 352, 352). B stands for batch size, and C stands for the number of weather variables. Channel progression is $64 \rightarrow 128 \rightarrow 256 \rightarrow 256 \rightarrow 512$.

The first convolutional layer transforms the input into 64 channels, followed by 4 residual blocks. Each of our residual blocks consists of two convolutional layers whose outputs are then processed by a squeeze and excitation block to reweight the channels based on their global importance. Each residual block has stride 2 to reduce the spatial dimension of the data. Then, the sum of the outputs and original inputs (modified to fit the dimension of the outputs) go through the ReLU activation function and are fed into the next residual block. For each convolutional layer, batch normalization is applied to the outputs before they are fed into the activation function. After the data has gone through all the residual blocks, global average pooling is applied to the final residual block output, which are then fed respectively into the continuous and binary heads for prediction. The continuous head has two linear layers and dropout (0.3) in between, while the binary head has two linear layers and outputs a logit. Details on model structure can be found in the flow chart below (Figure 1). We used a combined loss formula $loss = MSE(continuous) + 0.1 \times BCEWithLogits(binary)$.

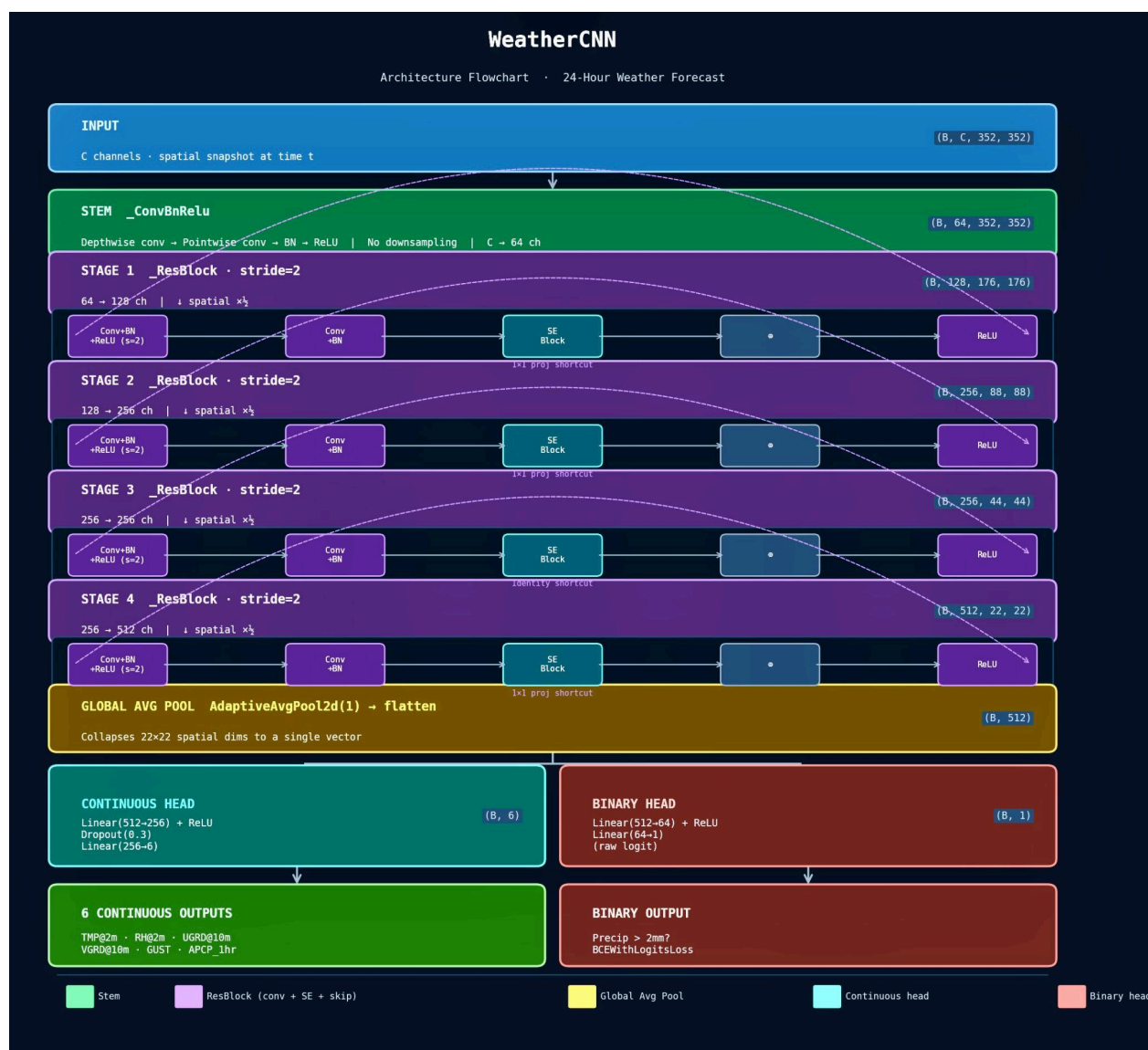


Figure 1

1.3 Hyperparameters

We trained our model with a batch size of 32, learning rate of $4e-4$ for 50 epochs on the Tufts HPC cluster. We used an Adam optimizer and gradients are clipped at 5. CosineAnnealingLR was used for learning rate decay.

1.4 Performance

The validation loss of the model began to plateau around epoch 30, and the best model appeared at epoch 39 with less than 0.25 validation loss. See figure 2 for the training and validation loss curves.

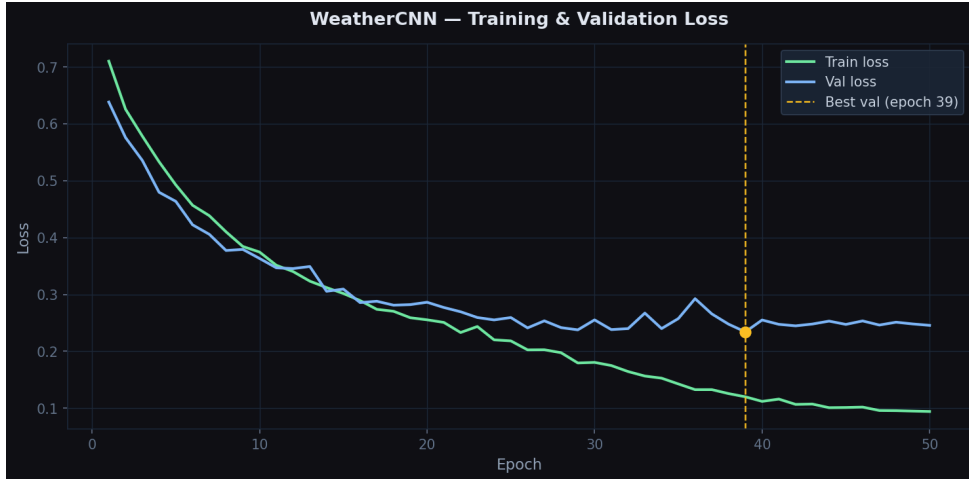


Figure 2

Table 1: Comparison of evaluation metrics compared to baseline model.

Weather Variable	Metric	Original Baseline (Stub)	Our New Model	Improvement
TMP @ 2m above ground	RMSE	4.8551	3.1937	-34%
RH @ 2m above ground	RMSE	23.0090	15.1353	-34%
UGRD @ 10m above ground	RMSE	3.7275	2.4979	-33%
VGRD @ 10m above ground	RMSE	2.8920	2.0930	-28%
GUST @ surface	RMSE	4.8696	3.6169	-26%
APCP (1hr acc fcst)	RMSE	4.6183	4.5350	-2%
APCP (> 2mm binary)	AUC	0.5062	0.7214	+43%

Our model performed significantly better than the baseline model in all metrics (table 1). 3 of the 6 continuous variable prediction RMSE were decreased by over 30% and 2 were decreased by over 25%. The AUC has also improved by 43%.

Part 2: Weather Diagnosis

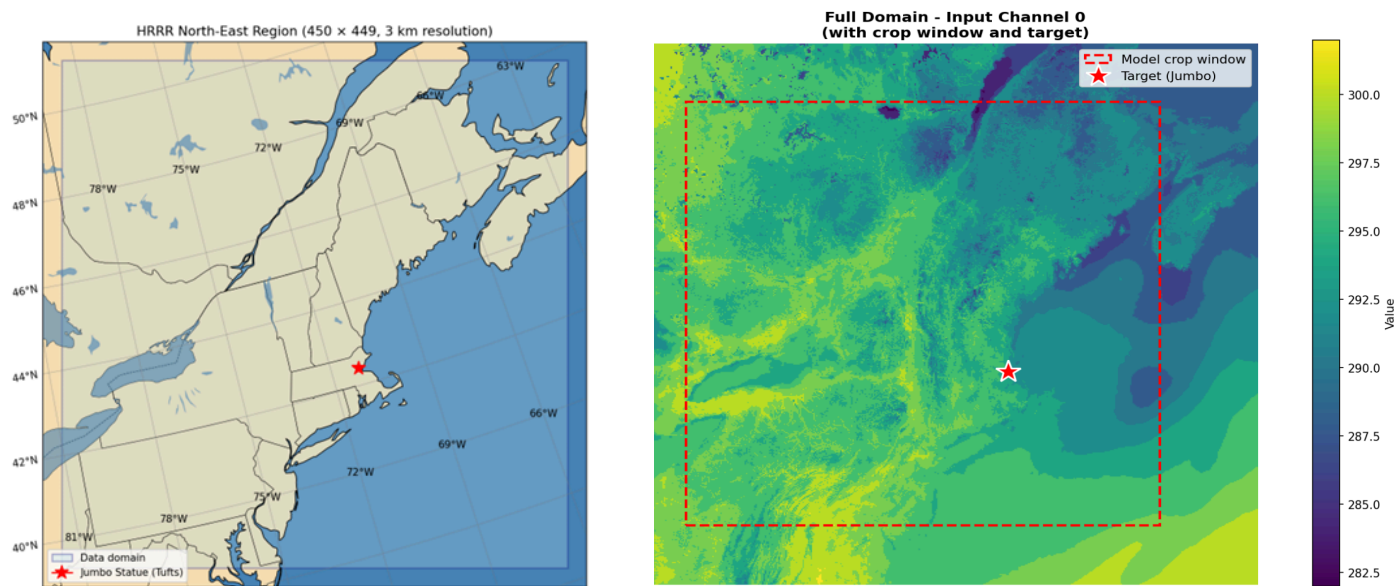


Figure 3: Left: HRRR Northeast U.S. domain with the target location (Jumbo statue at Tufts University, marked by the red star). Right: Example input field (Channel 0) with the model's cropped input window (red dashed box) centered around the target.

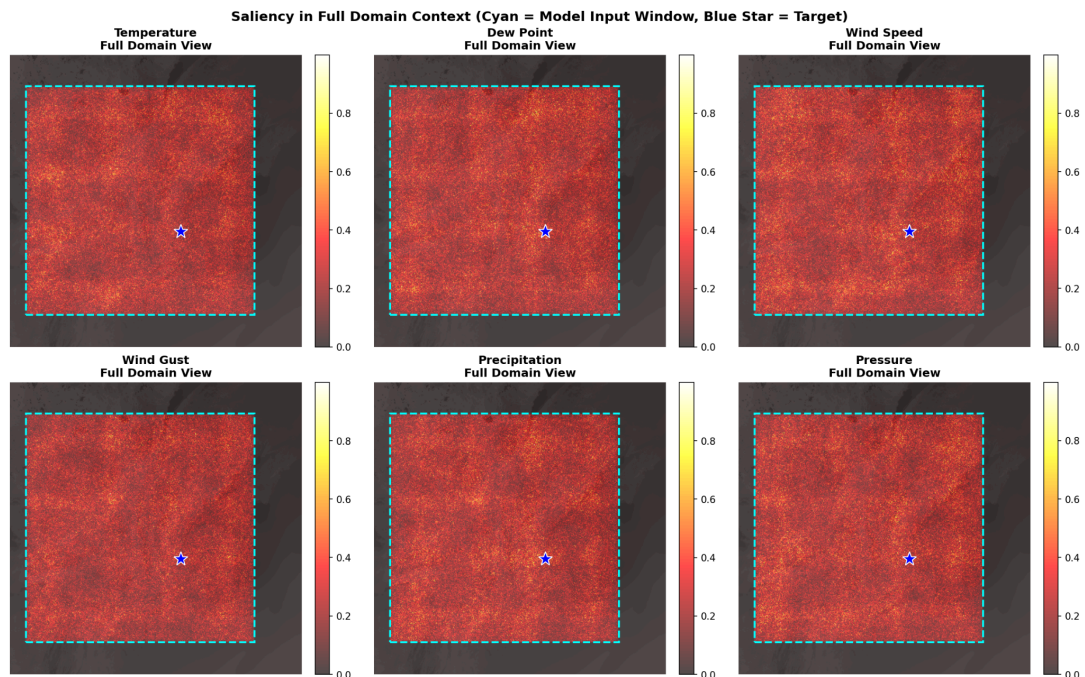


Figure 4: Saliency maps (gradient of output with respect to input) for Temperature, Dew Point, Wind Speed, Wind Gust, Precipitation, and Pressure are overlaid on the full spatial domain. The cyan dashed box indicates the model's input window, and the blue star marks the target location. Across all variables, sensitivity is broadly distributed throughout the input region rather than concentrated solely at the target point, indicating that the model relies on surrounding regional context to make its predictions.

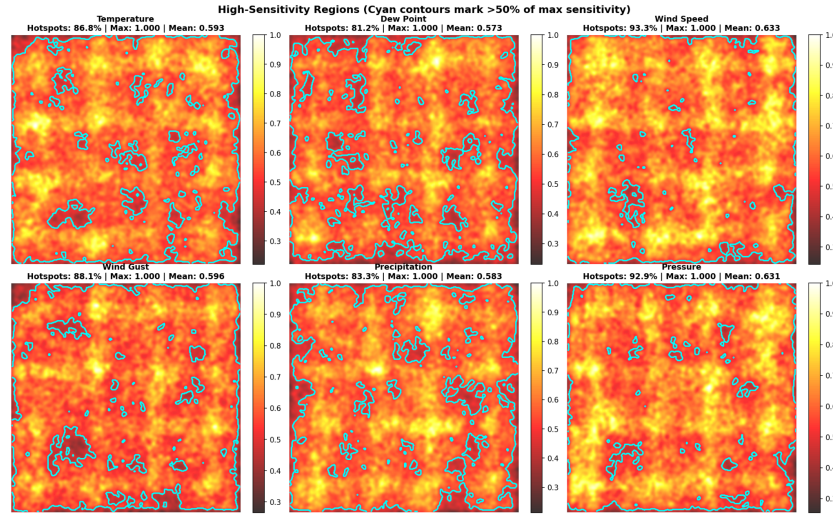


Figure 5: Saliency maps for each variable with cyan contours highlighting regions where sensitivity exceeds 50% of the variable’s maximum value. The reported hotspot percentages ($\approx 81\%$ – 93%) indicate that a large portion of the input window contributes meaningfully to each prediction. This demonstrates that the model’s sensitivity is spatially extensive rather than localized, with important information distributed across much of the domain for all variables.

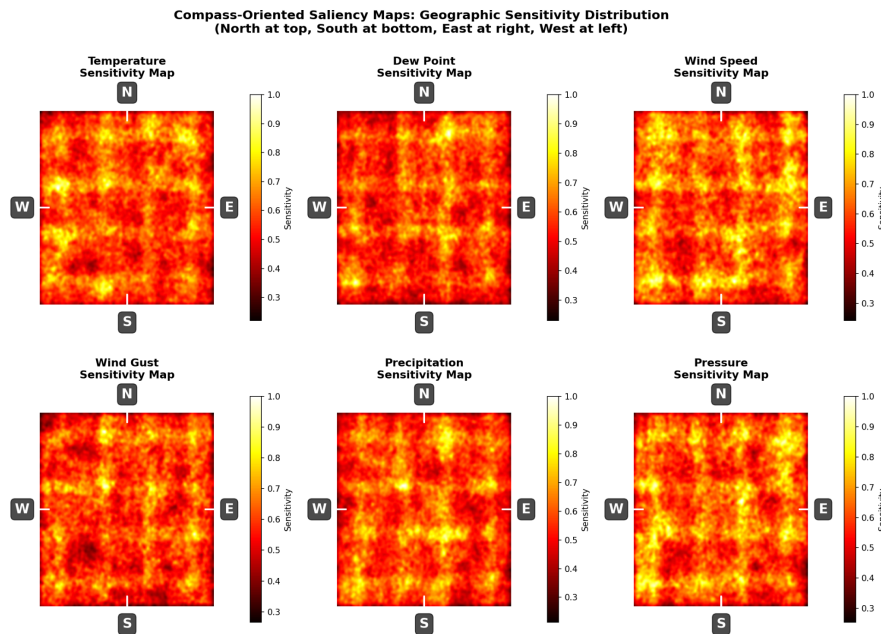


Figure 6: Saliency maps for each predicted variable displayed with geographic orientation (north at top, south at bottom, east at right, west at left). The color intensity represents the magnitude of sensitivity, indicating how strongly different regions of the input influence each prediction. Across all variables, sensitivity is distributed throughout the domain with no single dominant direction, though mild variations can be observed between regions. This visualization complements the spatial analysis by showing that the model integrates information from all geographic directions rather than relying on a single upstream source.

To start and understand which regions within the image influenced the model's prediction the most, we computed saliency maps to show the gradient of the output with respect to the input. Saliency maps were chosen because they highlight where small changes in the input affect the predicted values the most, allowing us to identify geographic areas that the model relies on most to make its prediction. From Figure 4, we can see that the model's sensitivity is not concentrated at the target location. Instead, we see that there is a spread throughout the entire region, indicating that the model is not making predictions solely based on local information; instead, it is using spatial context. This can be true because only looking locally would be inefficient since it can change drastically in 24 hours. This means we are currently learning a relationship between the target location and the surrounding regions, not specific points within the whole geographic area. We can further cement this notion by looking at Figure 5, where regions exceeding 50% of the maximum saliency cover a large portion of the input window (highlighted in cyan, making up about 80% of the input). This shows that a wide area of the map contributes meaningfully to each prediction, ignoring small parts but with no real correlation to those spots.

Looking at Figure 6, we can also analyze how sensitivity varies across geographic directions. While sensitivity is present in all directions, there are mild patterns in how the model weights the importance of different regions, especially depending on the source of information we are looking at. For example, for dew points, it seems like the northeast grid matters the most. This is true for several variables; some have slightly higher sensitivity in the western or southern portions of the map, but the differences are not large enough to raise concern or indicate high importance. It suggests that the model may be placing some importance on these regions while still using information from all directions regardless. Additionally if we also pay attention to Figure 3, we can interpret these directional patterns even further. For example, the western part of the input corresponds to the inland regions upstream of Boston, while the eastern side corresponds to the Atlantic Ocean. Since weather systems in the Northeast are typically affected by movement from west to east, there seems to be a slight increase in sensitivity in the western regions or in eastern corners, meaning that the model could be using this fact and taking into account upstream conditions to inform its predictions. Similarly, sensitivity in the southern regions may reflect taking into account the influence of warmer or more humid air rising from the south, which affects things like temperature and precipitation.

It is important to note, though, that these directional differences are relatively small and no single direction has dominance over the prediction of any of the variables, indicating again that the model is relying on the entire surrounding region since most of it exceeds 50% of the maximum saliency score. This makes sense because, in a region like Boston, all directions affect the weather: inland areas, ocean regions, and north-to-south movements. To conclude, it seems the gradients show us with certainty that the model is in a better place to be robust because it takes into account broad regional weather patterns, which aligns with the physical intuition of weather systems, where local conditions 24 hours into the future are influenced by large-scale structures spread over a large region.

Part 3: Independent Study

3.1 Feature Importance Study Methods and Hypothesis

We performed ablation experiments to determine which combinations of variables have the most predictive power. We retrain the CNN model from Part 1 with different combinations of features and see which setup leads to the best performance, or if subsets of the available features can reach the same performance as a model trained with all features. We hypothesize that a model trained on a carefully selected subset of variables can achieve performance comparable to a model trained on all available input channels. We evaluate this hypothesis by comparing evaluation metrics of each retrained model.

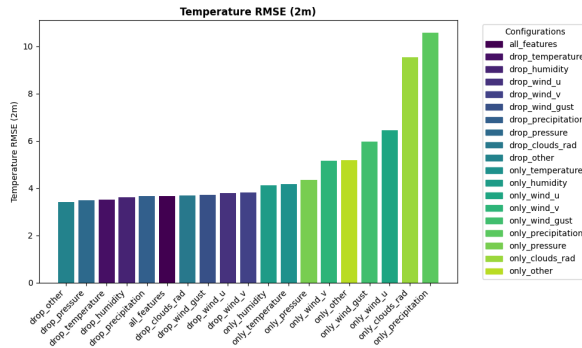
This experiment investigates the relative importance and redundancy of input variable groups in our CNN for weather prediction. We chose to first group the variables by type and then retrain CNNs with the same architecture with different configurations of the various groups.

Table 2: Feature group assignments and descriptions

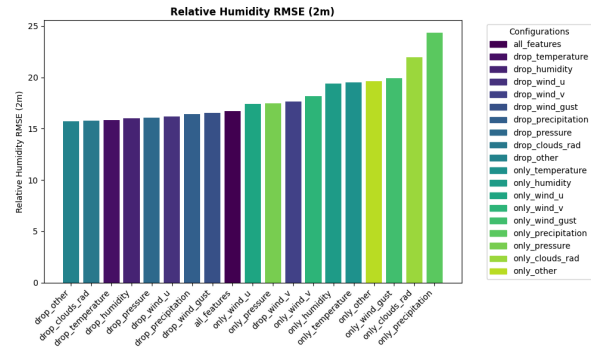
Group	# Channels	Features	Description
temperature	6	TMP@2m, TMP@1000mb, TMP@925mb, TMP@850mb, TMP@700mb, TMP@500mb	Air temperature at surface and multiple pressure levels; captures vertical thermal structure
humidity	7	RH@2m, DPT@1000mb, DPT@925mb, DPT@850mb, DPT@700mb, DPT@500mb	Moisture content via relative humidity and dew point; important for cloud formation and precipitation
wind_u	7	UGRD@10m, UGRD@1000mb, UGRD@925mb, UGRD@850mb, UGRD@700mb, UGRD@500mb, UGRD@250mb	Zonal (east-west) wind across atmospheric levels; drives horizontal transport
wind_v	7	VGRD@10m, VGRD@1000mb, VGRD@925mb, VGRD@850mb, VGRD@700mb, VGRD@500mb, VGRD@250mb	Meridional (north-south) wind; complements U component for full wind field
wind_gust	1	GUST@surface	Surface wind gust magnitude; captures extreme/local wind behavior
precipitation	1	APCP_1hr_acc_fcst@surface	Hourly accumulated precipitation; direct signal of rainfall
pressure	5	HGT@surface, HGT@1000mb, HGT@850mb, HGT@700mb, HGT@500mb	Geopotential height; represents large-scale atmospheric structure and pressure systems
clouds_rad	5	DSWRF@surface, TCDC, HCDC, MCDC, LCDC	Cloud cover (low/mid/high) and radiation; important for energy balance and precipitation
other	3	CAPE@surface, PWAT, VIL	Convective and integrated atmospheric indicators (instability, moisture, storm intensity)

We train two kinds of configurations: leave one group out and train on all remaining features, and train using just one group. This results in 19 unique configurations and we compare against the baseline of the CNN trained on all features. Experiments were trained with the same architecture as described above and in the years 2018-2020. Evaluation metrics are calculated on the test set consisting of 2021 data.

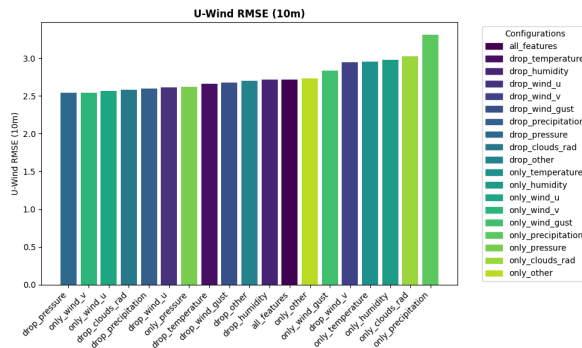
3.2 Feature Importance Study Results



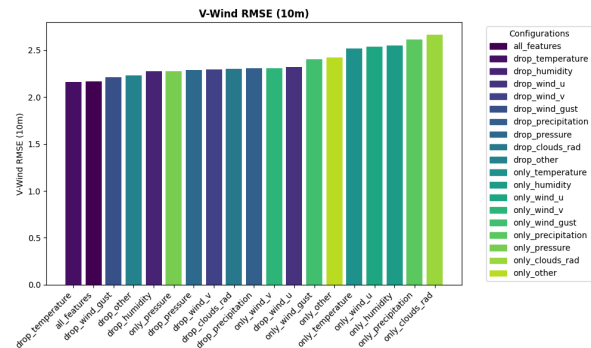
RMSE of TMP @ 2m above ground



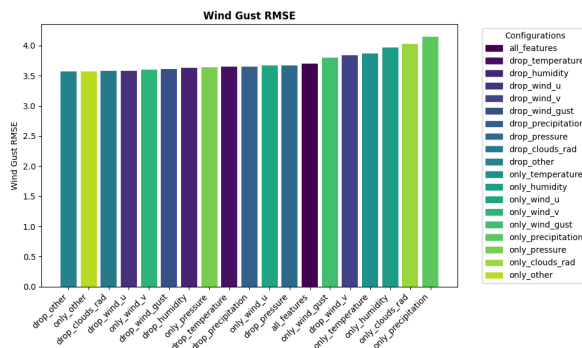
RMSE of RH @ 2m above ground



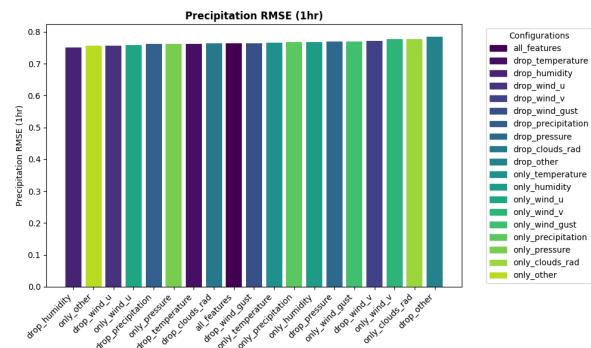
RMSE of UGRD @ 10m above ground



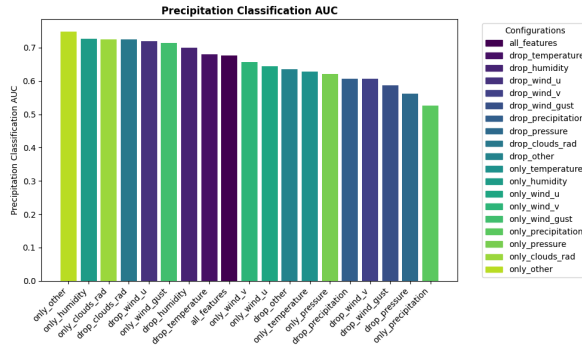
RMSE of VGRD @ 10m above ground



RMSE of GUST @ surface



RMSE of ACP (1hr acc fcst)



AUC of APCP (> 2mm binary)

Figure 6: Overall, training with single groups increases RMSE and decreases AUC. Training on the precipitation group gives the worst performance on most metrics. Dropping groups may slightly improve performance as well, suggesting that noise may be impacting the performance when training on all features.

Overall the ablation results support the hypothesis. Models with individual groups of features removed are able to achieve similar or better performance compared to a model trained on all features with the same training method. Notably the temperature, pressure, and ‘other’ groups may be introducing noise or redundancy rather than improving predictive performance. We can also observe that dropping wind groups results in performance worse than the baseline for some metrics, suggesting that it is an essential group of features, especially for the wind related targets. Additionally, the relatively stable performance across ‘drop one group’ experiments suggests that the model is able to compensate for missing information using correlated variables, which may indicate that many input features are redundant. These findings suggest that a smaller, carefully selected subset of meaningful variables can achieve comparable performance to the full feature set while potentially improving model efficiency and generalization.

3.3 Future Work

These results are informative for possibly removing features that do not help improve metrics for this weather prediction task. The benefit of removing features is faster training time and less computational power needed. Since weather prediction is location and time dependent with massive amounts of data being collected daily, improving the feasibility to retrain models with new data is worth further study. Future work should compare metrics of models trained on other combinations of the groups, not just leave one out or only one group to best determine the minimum set of features needed to effectively train a model for this task. Additionally, permutating which features are included in each group would be useful in determining if some features within groups are redundant.